

BridgeEDI — Multi-Country Adapter & Customer Workspace

Implementation Plan

Document type: Internal architecture & implementation plan (pre-development)
Platform: BridgeEDI (codebase: `zodiac/zodiac-api` + `zodiac/zodiac-front`)
Based on: Repository analysis + client/meeting requirements (customer workspace, country adapters, ERP ↔ Government flow)
Constraint: No code changes in this document — planning only

Executive Summary

BridgeEDI (Zodiac) already operates as an **invoice / EDI / tax-document middleware** with:

- Document intake (upload, API key, supplier token, SAP receive)
- Validation, conversion, correction
- Outbound connectors (PeppolSoft DBNA third-party, SAP billing POST for Mexico SAT/CFDI)
- Dashboards, certificates/mTLS, and Generative AI analytics over operational + SAP-shaped data

The business goal is **not** a greenfield rewrite. It is to **reuse the existing platform** so that:

1. **Each customer** gets an **isolated workspace** (their ERP, their transactions, their monitoring/AI).
2. **Each country** gets a **country adapter** containing only: mapping, validation, business rules, formatting, and government/endpoint integration.
3. **Shared platform** owns: auth, orchestration, storage, retries/monitoring, audit, notifications, and AI analytics over that customer’s data.

What exists today (verified)

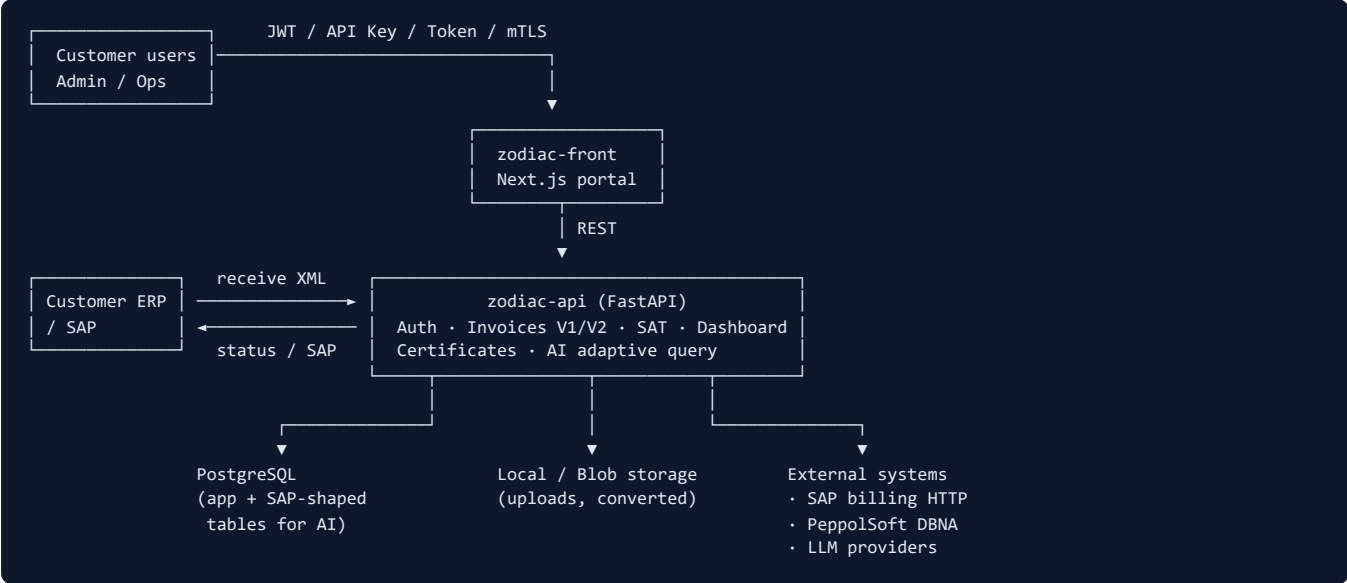
Capability	Status in repo
Receive invoices	Yes — V1, V2, SAT intake
Validate / map / transform / format	Yes — but embedded in services, not adapter interfaces
Send to government / partner endpoint	Partial — Mexico SAT → SAP and V1 → PeppolSoft ; live SAT.gov PAC APIs not found
Confirmation back to ERP	Partial — SAP response stored on SAT merges; V2 customer delivery is a TODO stub
Logs / monitoring / AI	Yes — dashboards + adaptive AI
Durable queue / Celery	Not present — FastAPI BackgroundTasks + in-memory status
Explicit country-adapter pattern	Not present — Mexico CFDI is a parallel module set
True multi-tenant <code>tenant_id</code>	Not present — isolation via <code>user_id</code> + <code>customer_id</code> + RFC lists

Recommended strategy

Extract the Mexico SAT/CFDI path into the **reference Country Adapter (MX)**, keep shared orchestration, then **clone the adapter contract** for the next country—without rebuilding auth, portal, storage, or AI.

Current Architecture

High-level (as implemented)



Three parallel invoice pipelines (FACT)

Path	Entry	Core services	Outbound
A — Legacy V1	/api/v1/invoices/process , /sap/process , /api/process	process_service.process_invoice_internal , format_router	external_api_service → PeppolSoft
B — Invoices V2	/api/v1/invoices-v2/upload , /sap/receive	validation → convert → deliver	customer_delivery_service (stub)
C — Mexico SAT/CFDI	/api/v1/sat/* , canonical/simple merge	sat_processor , merge services, sap_transformer	sap_api_client → SAP billing

There is **no single unified “country adapter” interface** today. Country/jurisdiction logic for Mexico is a **dedicated SAT stack**.

Folder Structure

Repository root (relevant)

Path	Purpose	Status
zodiac/zodiac-api/	Production FastAPI backend	Production
zodiac/zodiac-front/	Production Next.js portal	Production
zodiac/*.md	Docs / proposals / fix notes	Documentation
Root *.py (converter demos)	Ad-hoc XML/X12 scripts	Legacy / tooling — not wired into Zodiac runtime
Root zodiac-api/ stub	Incomplete duplicate	Dead / accidental

zodiac-api/app/

Folder	Purpose	Responsibilities	Dependencies	Status
api/	HTTP routers	Auth, invoices, V2, SAT, dashboard, certificates, customers	services, models, schemas	Production (sat_debug not mounted)
services/	Business logic	Process, SAT, SAP send/transform, AI SQL, certs, delivery	models, utils, config, DB	Production (+ large AI surface; delivery stub)
models/	ORM	Users, customers, invoices, SAT, certificates, AI memory	SQLAlchemy Base	Production
schemas/	Pydantic DTOs	Request/response validation	FastAPI	Production
utils/	Formatters/parsers	X12, EDIFACT, UBL, CFDI, PDF, crypto	—	Production
config/	Env + blob	Secrets, LLM knobs, storage mode	dotenv	Production
middleware/	mTLS helpers	Client cert header validation	certificates	Production-capable
tasks/	Ops scripts	Certificate expiration monitor	models	Ops script (cron not verified)
analysis/	Metrics/charts	Dashboard insight helpers	—	Production (AI/dashboard)
table_mapping/	SAP column metadata	AI SQL schema context	JSON files	Production (AI)

zodiac-front/src/

Folder	Purpose	Status
app/	Routes: dashboard, AI, invoices-v2, sat-documents, customers, upload	Production
components/	Feature UI (InvoicesV2, SAT, Dashboard, AI)	Production (some .bak leftovers)
lib/api.ts	Axios API client	Production
contexts/	AuthContext	Production
hooks/ , types/	Supporting	Supporting

Backend Architecture

Entry: app/server.py → FastAPI app , CORS, router registration, optional invoice-bot .env merge.

Database: PostgreSQL via DATABASE_URL , SQLAlchemy sync sessions, connection pool (pool_size default 5).

Auth surfaces:

- JWT (api/auth.py) — portal users
- API keys (api_key_auth.py) — machine invoice API
- Supplier tokens — SAT supplier intake
- Customer tokens + mTLS — delivery / secure channels
- Certificates lifecycle — api/certificates.py

Background work (FACT): FastAPI BackgroundTasks , asyncio.create_task for V1; **no Celery/Redis broker**. In-memory status maps exist (StatusTracker , _processing_invoices) — **fragile under multi-instance/serverless**.

Frontend Architecture

Stack: Next.js App Router, React, Tailwind, Axios, Recharts.

Customer-relevant UI surfaces:

- Admin: Dashboard V2, Intelligence/AI (/dashboard/ai), Invoices V2, SAT Documents, Customers, Certificates
- Customer user: /customer-invoices , /customer-sat-documents , /customer-dashboard

Meeting implication: Customer workspace needs a **dedicated portal space** (nav + data scoped to that customer), separate from the shared demo/ops analytics used today—without forking the entire app.

Integration Flow (Target vs Current)

Target (from meeting)

```
Customer ERP → BridgeEDI → Mapping/Validation/Rules/Format
→ Government Endpoint → Confirmation → BridgeEDI → Customer ERP
(+ logs, monitor, retry, AI)
```

Current closest match (Mexico SAT → SAP)

```
Supplier/Admin CFDI upload → sat_processor (parse/validate store)
→ simple/canonical merge → sap_transformer → sap_api_client POST
→ store sap_document_number / sap_response on merge record
```

Gap vs meeting: Confirmation is stored on BridgeEDI; **automatic push of confirmation back into customer ERP** is not implemented as a generic ERP callback service (SAP send response is captured locally). V2 “send to customer” is explicitly unfinished.

Current V1 third-party path

```
Upload → process_invoice_internal → format_router → external_api_service (PeppolSoft)
→ success/failed EDI tables
```

This is a **partner/network endpoint**, not a government tax portal.

Invoice Processing Flow (detailed)

Path A — V1 (process_service.process_invoice_internal)

Files: api/invoices.py , services/process_service.py , format_router.py , external_api_service.py , status_tracker.py , models invoice.py

Documented stages (in code comments/flow): Upload → XML validation → EDI conversion → EDI validation → 3rd party endpoint → DB save (with AI correction branches).

DB: zodiac_invoice_success_edi , zodiac_invoice_failed_edi (+ business data helpers).

Path B — V2

Files: `api/invoices_v2.py` , `invoice_v2_validation_service.py` , `invoice_v2_correction_service.py` , `invoice_conversion_service.py` , `converted_invoices.py` , `customer_delivery_service.py`

Flow: Document row → validate → validated row → convert to customer `target_format` → (intended) deliver.

DB: `v2_invoice_documents` , validated tables, converted invoices, BI tables.

Path C — SAT/CFDI (current country implementation)

Files:

API: `sat.py` , `sat_canonical.py` , `sat_simple_merge.py` , `sat_supplier_mapping.py`

Services: `sat_processor.py` , `sat_canonical_merge_service.py` , `sat_supplier_mapping_service.py` , `sap_transformer.py` , `sap_api_client.py` , `sap_send_all.py`

Utils: `cfdi_parser.py` , `xml_to_cfdi.py`

Models: `sat_document.py` , `sat_simple_merged.py` , `sat_canonical_merged.py` , `sat_supplier_account_mapping.py` , `customer_receiver_rfc.py`

Flow: Intake CFDI XML → parse (SAT namespaces) → store → merge → transform to SAP JSON → HTTP to SAP → persist confirmation fields.

SAP Integration

Concern	Implementation	Notes
Inbound invoice XML from SAP	<code>POST /invoices-v2/sap/receive</code> , V1 <code>/invoices/sap/process</code>	Multipart / process pipeline
Outbound billing to SAP	<code>SAPAPIClient.send_json_to_sap_with_session</code>	CSRF + session POST; URL/credentials hardcoded in source (security risk)
CFDI → SAP JSON	<code>sap_transformer.py</code>	Country-specific transform for MX
Bulk send	<code>sap_send_all.py</code>	Pending merges
AI over SAP tables	<code>sap_sql_agent.py</code> , <code>table_mapping/*</code> , adaptive query	Uses app <code>DATABASE_URL</code> / <code>AI_CONTEXT_SOURCE</code> — not a separate live SAP RFC layer for invoices

Unverified: Whether production overrides hardcoded SAP URL via env (not evident in `sap_api_client.py` from analysis).

Government Integration

Finding	Detail
Implemented “government-adjacent” stack	Mexico CFDI intake + RFC mapping + merge + SAP posting
Live SAT.gov / PAC / timbrado APIs	Not found in codebase
“Government confirmation” today	Practically = SAP accept/reject response after CFDI-derived posting, plus document status in portal
PeppolSoft	Third-party EDI network, used by V1

For the new customer, the meeting states a **different HTTP/REST endpoint** and **3–4 document types** with confirmation returned to BridgeEDI then ERP. That endpoint **is not in the repo** — must be supplied as integration inputs (OpenAPI, auth, payloads, callbacks).

Current Country Adapter (Mexico — de facto)

Even without an adapter interface, Mexico is the **reference country module**:

Adapter concern	Where it lives today
Mapping	<code>sap_transformer</code> , supplier/GL mapping services, RFC tables
Validation	<code>CFDIParser</code> , SAT processor validation, receiver RFC allow-lists
Business rules	Merge rules (simple/canonical), duplicate checks, period/fiscal grouping
Formatting	CFDI XML parse; SAP JSON output format
Endpoint integration	<code>sap_api_client</code> / <code>sap_send_all</code>

Everything else (auth, UI shell, DB engine, dashboards, AI chat, certificates) is shared platform.

Shared Components

Layer	Components (reuse as-is or with thin wrappers)
Auth	JWT, API keys, supplier/customer tokens, certificates/mTLS
Portal shell	MainLayout, Sidebar, AuthContext, api.ts patterns
Orchestration primitives	BackgroundTasks patterns, status tracking (needs hardening)
Storage	<code>file_storage_service</code> , uploads/converted/Blob
Customer master	<code>Customer</code> , <code>UserCustomer</code> , target_format
Monitoring UI/API	Dashboard V2 endpoints + components
AI	Adaptive query, charting, chat thread store (scoped by user today)
Ops	Health endpoints, logging

Customer Workspace Design

Meeting requirement

Exclusive space for the customer to run their own transactions; not mixed with other demo/scenario data.

Current isolation (FACT)

- Documents often filtered by `user_id`
- Customer users via `UserCustomer`

- Mexico: `CustomerReceiverRfc` for allowed RFCs
- **No** first-class `workspace_id` / `tenant_id`

Proposed model (design — not implemented)

```
Workspace (Customer)
├── Users (roles)
├── ERP connection config (base URL, auth, callback)
├── Enabled country adapters [MX, XX, ...]
├── Credentials / certificates
└── Transaction + audit data (scoped)
```

Reuse: Existing `Customer` + `UserCustomer` as the workspace seed.

Add (planned): workspace-scoped routing to country adapters; stricter query filters; optional dedicated nav/branding for customer portal.

AI: Feed AI only from **that workspace’s** operational tables (meeting: real-time transactional data into AI analysis)—requires enforcing `customer_id` / `workspace_id` in adaptive SQL paths (today AI often sees broader schema).

Country Adapter Design

Contract (proposed)

Each country adapter package exposes:

Hook	Responsibility	MX reference today
<code>parse_and_validate(payload)</code>	Schema + tax-id rules	<code>CFDIParser</code> + <code>sat_processor</code>
<code>apply_mappings(ctx)</code>	Field / GL / party mapping	supplier mapping + transformer
<code>apply_business_rules(ctx)</code>	Accept/reject/merge rules	merge services
<code>format_outbound(ctx)</code>	Target government/ERP payload	SAP JSON
<code>send(ctx)</code>	Call endpoint	<code>SAPAPIClient</code>
<code>handle_confirmation(ctx)</code>	Normalize response	<code>sap_response</code> fields
<code>push_to_erp(ctx)</code>	Confirmation to customer ERP	Gap — to build

Shared platform owns

Intake API → authenticate → load workspace → select adapter by country → run hooks → persist audit → update monitor → notify → AI indexing.

Adapter isolation rule (from meeting)

Adapter code **must not** be reused across countries. Shared utilities (HTTP client, retry policy, audit writer) stay in core; **rules/mappings/formats/endpoints** stay in the adapter.

AI Integration

Current	Role
/dashboard/ai , POST /api/query/adaptive	NL analytics over DB
Dashboards	Operational KPIs
Chat threads	Optional persistence (ada_* / schema-chat)

Meeting intent: AI shows **realtime analysis of customer transaction data**, not a separate product.

Plan implications:

1. Scope AI queries to workspace data.
2. Reuse adaptive pipeline + charts.
3. Add operational metrics events (submit, confirm, fail, retry) as first-class facts for AI.
4. Do **not** let AI mutate government payloads without controlled workflow.

Monitoring Architecture

Existing	Gap for new program
Dashboard V2 inbound/outbound/business	Workspace-scoped views
Failed invoice analysis	Adapter-level failure reasons
Certificate health	Per-workspace certs
In-memory status for V1	Durable transaction timeline

Required for meeting SLAs: correlation ID end-to-end, retry visibility, endpoint health, alert hooks (email/Slack partially TODO in cert monitor).

Security

Control	Current	Plan note
JWT / RBAC	Yes	Extend roles for workspace
API keys / tokens	Yes	Per-workspace keys
mTLS / certificates	Yes	Per-customer ERP channels
Secrets	Mixed (env + hardcoded SAP/Peppol)	Must move outbound creds to vault/env before customer go-live
TLS verify disabled on SAP client	FACT in code	Fix before production customer use
Audit	Partial (SAT logs, invoice tables)	Standardize transaction event log

Multi-Tenant Strategy

Phase approach:

- 1. **Logical tenancy** on existing Postgres (workspace_id on transaction tables; filter everywhere).
- 2. **Config tenancy** (ERP endpoints, adapter enablement, mappings).
- 3. Later: stronger isolation (schema-per-tenant / DB-per-tenant) **only if** compliance requires — **not** evidenced as required in current code.

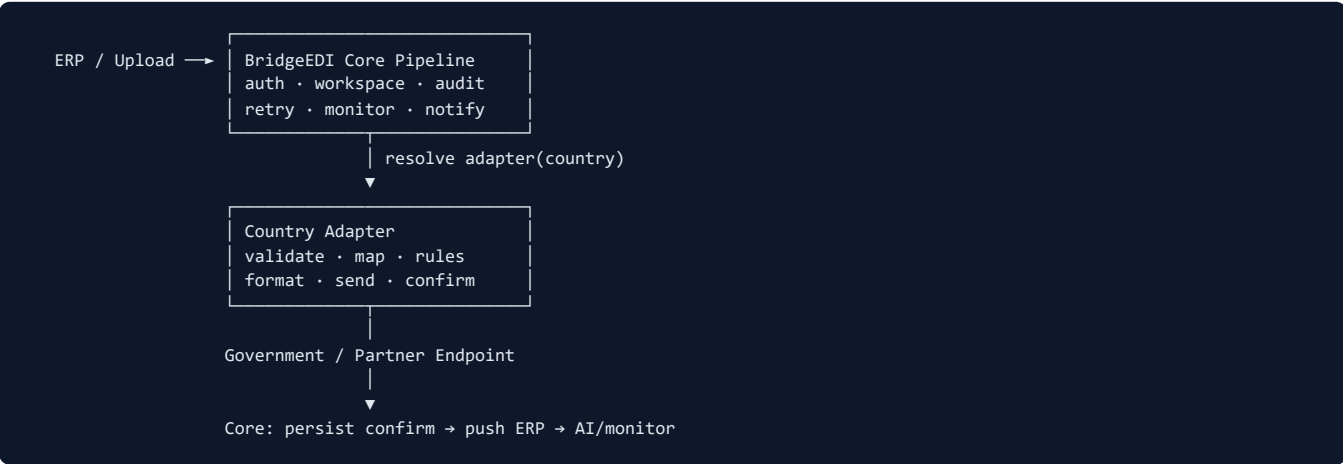
Risk if skipped: AI or dashboards leaking cross-customer rows (possible today if filters incomplete — full audit **not** exhaustively verified).

Recommended Folder Structure

```
zodiac-api/app/
core/                # NEW thin layer (orchestration, retry, audit interfaces)
  workspace/
  pipeline/
  adapters/base.py   # Protocol / ABC only
adapters/
  mx_cfdi/           # EXTRACT from sat_* + sap_transformer + sap_api_client pieces
    mapping/
    validation/
    rules/
    formatting/
    connector/
  <country_code>/    # NEW adapters clone this layout only
api/                 # Keep; add workspace-scoped routes gradually
services/            # Shared services remain; country logic moves out over time
...
zodiac-front/src/
  app/workspace/[customerId]/ # NEW customer space routes (or reuse customer-* with stricter scope)
  adapters/            # Optional UI panels per country
```

Important: Prefer **incremental extraction** of MX into adapters/mx_cfdi over a big-bang move, to limit regression on live SAT customers.

Recommended Adapter Pattern



Step-by-Step Development Plan

Phase 0 — Discovery freeze & inventory (1–2 weeks)

Objectives	Document MX reference adapter boundaries; list customer ERP + government API contracts; inventory hardcoded secrets
Files involved	SAT/SAP/V2/V1 services listed above; <code>andy.txt</code> requirements
Dependencies	Client OpenAPI/auth samples; ERP callback contract
Complexity	Low–Medium
Risks	Incomplete endpoint specs delay Phase 2
Regression	None (docs only)
Outcome	Adapter interface draft + gap list signed off

Phase 1 — Customer workspace MVP (2–3 weeks)

Objectives	Isolated customer space in portal; data scoped to customer; enable reuse of invoices + monitoring UI
Files involved	<code>Customer</code> , <code>UserCustomer</code> , customer portal pages, dashboard filters, <code>api.ts</code>
Dependencies	Phase 0 tenancy fields decision
Complexity	Medium
Risks	Missed SQL filters → data leak
Regression	Admin global dashboards must keep working
Outcome	Demo-able “customer exclusive space” for meeting

Phase 2 — Extract MX as reference Country Adapter (3–5 weeks)

Objectives	Introduce adapter interface; wrap existing SAT→SAP path behind it; shared pipeline calls adapter hooks
Files involved	<code>sat_*</code> , <code>sap_transformer</code> , <code>sap_api_client</code> , new <code>adapters/mx_cfdi</code> , thin <code>core/pipeline</code>
Dependencies	Phase 1 workspace context passed into pipeline
Complexity	High
Risks	SAT regression for existing Mexico flows
Regression	High on SAT send/merge — require golden tests on CFDI fixtures
Outcome	Same MX behavior via adapter API; proof that “new country = new adapter package”

Phase 3 — Durable processing & retries (2–3 weeks)

Objectives	Replace in-memory status with DB-backed transaction log + retry policy; optional queue (Redis/SQS/Service Bus)
Files involved	<code>status_tracker</code> , <code>invoices.py</code> background tasks, new transaction/outbox tables
Dependencies	Infra choice (meeting: must handle 1k–10k invoices smoothly)
Complexity	High
Risks	Under-estimating serverless multi-instance issues
Regression	V1 status polling UX
Outcome	Reliable retries/confirmations under load

Phase 4 — New country adapter (customer scenario) (3–6 weeks)

Objectives	Implement adapter for target country: mapping, validation, rules, format, endpoint send, confirmation handling
Files involved	New <code>adapters/<cc>/</code> ; workspace ERP connector config; confirmation → ERP push
Dependencies	Government API credentials; 3–4 document schemas; ERP update API
Complexity	High (integration-bound)
Risks	Spec drift; auth differences (OAuth vs API key vs mTLS)
Regression	Should be isolated if adapter boundary is clean
Outcome	End-to-end: ERP → BridgeEDI → Gov → Confirm → ERP

Phase 5 — ERP confirmation round-trip (2–3 weeks)

Objectives	Generic <code>push_to_erp</code> with per-workspace HTTP config; map government confirmation to ERP status fields
Files involved	New ERP connector service; replace/extend delivery stub patterns
Dependencies	Customer ERP API
Complexity	Medium–High
Risks	Idempotency / duplicate status posts
Regression	Existing SAP send path
Outcome	Meeting flow fully closed

Phase 6 — Workspace-scoped AI & monitoring (2–3 weeks)

Objectives	AI + dashboards read only workspace transactions; realtime feed of pipeline events
Files involved	<code>adaptive_query</code> , dashboard V2, Intelligence UI, event writers from pipeline
Dependencies	Phases 1–4 data model
Complexity	Medium
Risks	AI SQL escaping tenant filters
Regression	Global admin AI analytics
Outcome	“Analysis based on what is already there” for the customer

Phase 7 — Hardening & go-live (2 weeks)

Objectives	Secrets vault, TLS verify, load test 10k invoices, runbooks, alert channels
Files involved	<code>sap_api_client</code> , <code>external_api_service</code> , config, cert monitor TODOs
Dependencies	Staging environment
Complexity	Medium
Risks	Production credential rotation
Regression	Connector auth
Outcome	Customer-ready platform path

Complexity & timeline note (from meeting)

Previous direct integrations ~**10 days**. Platform introduction must **not** inflate timelines for the same integration scope.

Implication: Phases 1–2 (workspace + MX extraction) are **platform investment**; Phase 4 (new country) should approach historical speed **once** adapter scaffolding exists. Show the client a **workspace + MX-via-adapter demo** early; treat new-country endpoint work as the variable path.

Inputs still required (cannot be verified from repo)

1. Target **country** and **government API** OpenAPI / auth / callbacks
2. **Document types** (3–4) schemas and samples
3. **Customer ERP** endpoints for submit + confirmation update
4. Non-functional targets (latency budget vs direct integration)
5. Whether confirmation is sync HTTP response only or also async webhook
6. Whether PeppolSoft remains in scope for this customer (meeting focuses on government endpoint)

Risk Register

Risk	Severity	Mitigation
Rewriting instead of extracting MX	High	Adapter wraps existing SAT/SAP path first
Cross-customer data exposure	Critical	Workspace filters + AI scoping tests
In-memory queues under scale	High	Phase 3 durable jobs before 10k volume claims
Hardcoded SAP credentials / SSL verify=False	Critical	Env/vault + TLS before customer prod
Customer delivery stub	High	Phase 5 ERP push
Spec unknown for new government API	High	Block Phase 4 until samples arrive
Timeline inflation	Medium	Parallelize workspace UI with MX extraction; reuse converters

Success Criteria

1. **Same BridgeEDI platform** serves the new customer workspace.
2. **Mexico path** still works, ideally behind adapter interface.
3. **New country** added primarily as a new adapter package + config.
4. Flow demonstrated: **ERP → BridgeEDI → Endpoint → Confirmation → ERP**.
5. Monitoring + AI show **that customer's** transactions.
6. Volume handling does not rely on single-process memory alone.

Closing

The repository already contains a working middleware for invoice/tax document processing—especially a complete **Mexico CFI → SAP** vertical and shared portal/AI/security. The multi-country program should **formalize what Mexico already is (an adapter)** and **add customer workspaces**, then **plug the next country's mapping/validation/rules/format/endpoint** into the same core pipeline.

This plan is the prerequisite for development start; no application code was modified to produce it.

Document: *IMPLEMENTATION_PLAN.md*

Platform: *BridgeEDI / Zodiac*

Document: *IMPLEMENTATION_PLAN.md* · Platform: *BridgeEDI / Zodiac* · Planning only — no code changes